ADVANCED MODULE

The Intentional Vibe Coder

A Framework for Disciplined AI-Assisted Development

Core Concepts

- The 3-Question Filter
- 5 Anti-Patterns of Vibe Coding
- The Healthy Vibe Coding Loop

ASSIGNMENT

Apply the 3-Question Filter
to your MVP backlog
& kill features that don't pass

CHECK-IN

Your AI Coding Experience So Far

1. **Your Tool**

   Which AI coding assistant do you use?

2. **Biggest Win**

   What's the best thing you've built with AI?

3. **Biggest Fail**

   Where did an AI project go off the rails?

Share with your neighbor -- 2 minutes each

RECAP

Where We Are in the Course

1. You've validated a real problem worth solving
2. You've built (or started building) an MVP prototype
3. Today: Making sure AI accelerates you -- not derails you

AI removed the implementation bottleneck. The decision bottleneck remains.

SECTION A

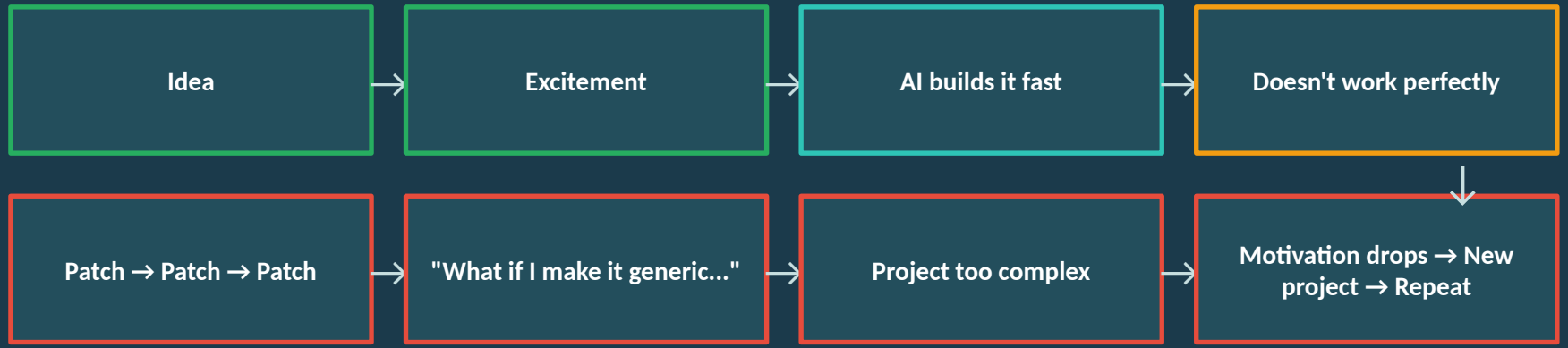
The Core Problem

Why AI-assisted projects explode in complexity and die

SECTION A

The Vibe Coding Death Spiral

AI copilots removed the bottleneck of implementation speed but NOT the bottleneck of decision quality.



"I can build it" stopped being a good reason to build it.

The question is no longer CAN you build it.
It's SHOULD you build it.

SECTION B

The 3-Question Filter

Before starting ANY feature, ask all three

SECTION B

The 3-Question Filter

1. **Has someone asked for this?**
   Validates real demand vs. imagined need. If you're the only user and you haven't needed it yet -- you don't need it.

2. **Does this make the product 10x better?**
   Ensures you're building differentiation, not parity. Your competition is a spreadsheet or doing it manually.

3. **Would I build this by hand?**
   The maintenance honesty check. AI code costs near-zero to write but full price to maintain and debug.

All 3 Yes → Build it | Any No → Capture the idea, move on

SECTION B

Decision Matrix in Practice

Feature	Asked for?	10x better?	Build by hand?	Verdict
User-requested export to CSV	✓	✓	✓	BUILD
Generic plugin system for v1	✗	✗	✗	SKIP
Custom field types (like Notion)	✗	✗	✗	SKIP
One-click deploy for your niche	✓	✓	✓	BUILD
Dark mode	✓	✗	✗	SKIP

If in doubt, don't build. Capture the idea in a backlog and move on.

SECTION C

The 5 Anti-Patterns of Vibe Coding

Patterns that kill AI-assisted projects

SECTION C

Anti-Patterns 1 & 2

1 The Generalization Trap

"If I make the data model generic, it can handle 100x more use cases"

Reality: The more generic your tool, the less useful it is for each specific case.

Rule: Earn generalization through multiple successful specific applications.

2 Infrastructure Before Users

Building plugin systems, SDKs, credential stores, multi-tenancy before anyone uses v1

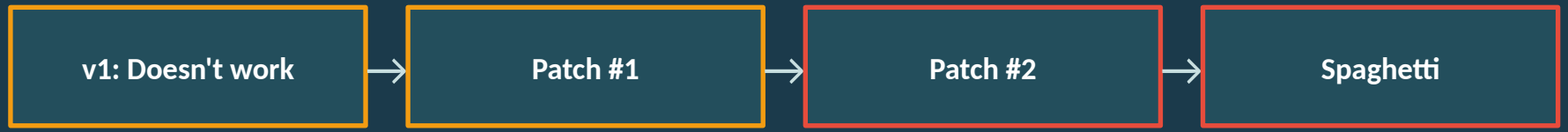
Reality: Infrastructure for hypothetical users is waste.

Rule: Build infrastructure only when concrete usage demands it.

SECTION C

Anti-Pattern 3: The Patch Spiral

Feature doesn't work → patch → patch → patch → spaghetti



Two failed attempts means the design is wrong, not the implementation.

Rule: Doesn't work after 2 tries? Delete everything. Redesign from scratch.

SECTION C

Anti-Patterns 4 & 5

4 Complexity Blindness

Needing extensive docs / memory files to remember your own project's gotchas

If you need a cheat sheet to work on your own code, it's too complex.

Rule: Simplify until a fresh dev can understand it without context docs.

5 The AI Yes-Man Problem

AI builds everything you ask for, never pushes back on scope

AI optimizes for "request fulfilled", not "project stays maintainable".

Rule: You are the PM. The AI is the contractor.

SECTION D

The Healthy Vibe Coding Loop

From chaotic building to intentional shipping

SECTION D

The Healthy Loop

Idea

Who is this for?

→

For whom?

Validate demand

→

Smallest feature

Minimal scope

↓

Iterate or discard

Based on signal

→

Show 1 user

Get real feedback

→

Build yourself

AI as reviewer only

External feedback replaces internal excitement as the signal

SECTION D

Key Shifts

AI is the builder	→	**YOU are the builder, AI is the advisor**
Ship everything you can build	→	**Ship the smallest thing that tests your hypothesis**
Internal excitement = signal	→	**External feedback = signal**
Code is cheap to generate	→	**Every line is maintenance debt that feels free**

SECTION E

Practical Rules

Concrete habits for intentional AI-assisted development

SECTION E

5 Practical Rules

1	**Feature Budget**	Maximum 1 new feature per week. Build, deploy, get feedback, iterate.
2	**LOC Honesty**	AI-generated code counts double as debt. Cheap to write, expensive to understand.
3	**Post-Session Review**	After every AI session: git diff --stat. Files you didn't request? Ask why, or revert.
4	**The Post-It Test**	"This tool does [one thing] for [one person]." Every feature must serve that sentence.
5	**Deletion Courage**	Deleting code is a feature. Every line removed is maintenance saved.

Discipline beats speed. Every time.

This tool does

[one thing]
for [one person]

Every feature must serve this sentence.
Put it on a Post-It. Tape it to your monitor.

DISCUSSION

Questions to Explore

1. How does AI copilot availability change the economics of software decisions?
2. When does "I can build it" stop being a good reason to build it?
3. How do you maintain architectural ownership when AI writes most of the code?
4. What's the relationship between implementation speed and decision quality?
5. How do you build the discipline to say "no" when building is effortless?

Break into groups of 3-4. Pick two questions. 10 minutes.

WORKSHOP

Apply the Framework to Your MVP

1	**List your features**	Write down every feature you've built or planned for your MVP
2	**Run the 3-Question Filter**	For each feature: Asked for? 10x better? Build by hand?
3	**Kill ruthlessly**	Any feature with a "No" -- move to backlog or delete
4	**Write your Post-It**	"This tool does [one thing] for [one person]." Share with the group.

KEY TAKEAWAYS

What to Remember

1. AI removed implementation friction -- but not the need for discipline.
2. Every feature must pass the 3-Question Filter before you build it.
3. The 5 anti-patterns are the enemy: generalization, premature infra, patching, complexity, the yes-man.
4. You are the PM. AI is the contractor. Never let the contractor decide what to build.
5. Deletion is a feature. Simplicity is the ultimate sophistication.

Build less.
Build better.
Ship what matters.

Now go apply the filter
to your own project.